

Module I: Introduction

1.1 What is Natural Language Processing (NLP)

Natural Language Processing (NLP) is a subfield of Artificial Intelligence (AI) and computational linguistics that focuses on enabling computers to understand, interpret, process, and generate human languages in a meaningful way. Unlike programming languages, natural languages (such as English, Hindi, or Kannada) are complex, ambiguous, and often context-dependent. NLP bridges the gap between human communication and machine understanding, allowing systems to interact with users in their own language.

Natural Language Processing (NLP) is concerned with the development of computational models of aspects of human language processing. There are two main reasons for such development:

1. To develop automated tools for language processing
2. To gain a better understanding of human communication

Building computational models with human language-processing abilities requires a knowledge of how human acquire, store and process language. It also requires a knowledge of the world and of language.

The scope of NLP ranges from basic language tasks—like tokenization, part-of-speech tagging, and parsing—to advanced applications such as machine translation, sentiment analysis, question-answering, speech recognition, and text summarization. NLP draws heavily from multiple disciplines:

- Linguistics: to understand grammar, syntax, semantics, and pragmatics.
- Computer Science: to design algorithms and models for processing text and speech.
- Statistics and Machine Learning: to learn patterns from large datasets for probabilistic language modeling.

Two Major Historical Approaches to NLP

1. Rationalist (Rule-Based / Symbolic) Approach

- **Description:** In the early stages of NLP (1950s–1980s), systems were built using **hand-crafted linguistic rules** created by experts. These rules encoded grammatical structures, morphological variations, and semantic relationships.
- **In NLP:** Language is modeled using **explicit rules and formal grammars** created by linguists and domain experts. The system's ability to understand and generate language comes from logically applying these rules.
- **Example:**
 - Using **Context-Free Grammar (CFG)** to parse sentences.
 - Early NLP systems like ELIZA, SHRDLU.

- **Strengths:**
- Produces predictable and interpretable outputs.
- Works well in controlled, well-defined domains.
- **Characteristics:**
 - Deterministic in nature — produces predictable results.
 - Relies on explicit **lexicons** (dictionaries) and **grammar rules**.
 - Examples: ELIZA (1966), rule-based machine translation systems.
- **Limitations:** Time-consuming to develop, not scalable for all languages, and unable to handle ambiguous or ungrammatical input effectively.

2. Empiricist (Statistical /Corpus-Based) Approach

- **Description:** Emerged in the late 1980s–1990s, leveraging **large language corpora** and **probabilistic models**. Instead of hand-coded rules, the system learns language patterns from data using statistical methods such as Hidden Markov Models (HMMs), n-grams, and probabilistic parsers.
- **Characteristics:**
 - Data-driven — adapts from examples rather than strict rules.
 - More flexible in handling noisy or ambiguous input.
 - Examples: Statistical machine translation (e.g., IBM Models), speech recognition systems.
- **In NLP:** Language processing relies on **large corpora** and statistical models to infer probabilities of words, phrases, and structures.
- **Example:**
 - **N-gram models** predicting the next word based on previous ones.
 - Statistical machine translation (IBM Models).
- **Strengths:**
 - Adapts to variations in language use.
 - Can handle noisy and ambiguous input better than rigid rule-based systems.
- **Limitations:** Requires large annotated datasets and may fail when encountering patterns absent in training data.
- **Key Difference:**
- **Rationalist** → Language knowledge is innate and modeled through rules.
- **Empiricist** → Language knowledge is acquired from experience and modeled through data.

One of the biggest challenges in NLP is ambiguity—the same word or sentence can have multiple meanings depending on context. For example, the word “bank” could refer to a financial institution or the side of a river. NLP systems address these challenges using rule-based approaches, statistical methods, or deep learning models.

In real-world applications, NLP plays a vital role in search engines, chatbots, virtual assistants, information retrieval systems, and multilingual communication tools. Its importance continues to grow as digital communication expands globally, making it a cornerstone technology for human–machine interaction.

1.2 Origins of NLP

The origins of NLP can be traced back to the mid-20th century, closely tied to the development of Artificial Intelligence (AI), linguistics, and computer science. One of the earliest conceptual foundations was laid by Alan Turing in his 1950 paper “Computing Machinery and Intelligence”, where he proposed the Turing Test as a measure of a machine’s ability to exhibit human-like intelligence through language interaction. This idea highlighted the importance of natural language as a key indicator of intelligence.

In the 1950s and 1960s, the first practical NLP systems emerged, primarily as rule-based translation programs. The Georgetown-IBM experiment (1954) was a landmark demonstration, where a computer translated over 60 Russian sentences into English using a set of hand-crafted grammatical and lexical rules. Around the same time, programs like ELIZA (1966) by Joseph Weizenbaum simulated conversation using pattern matching and substitution rules, giving the illusion of understanding.

Early NLP research was heavily influenced by the Rationalist approach, focusing on symbolic processing and linguistic rules derived from formal grammar theories, such as those proposed by Noam Chomsky. In the late 1980s and 1990s, however, the Empiricist approach gained momentum, fueled by the availability of large text corpora and advances in statistical modeling. This shift led to the development of probabilistic methods, such as Hidden Markov Models (HMMs) for speech recognition and statistical machine translation systems.

Over time, NLP evolved through three major eras: rule-based systems, statistical methods, and the modern deep learning era, which began in the 2010s. Today’s NLP combines computational power, big data, and neural networks to achieve human-level performance in tasks like translation, summarization, and question answering—marking a long journey from its modest, experimental beginnings in the 1950s.

1.3 Language and Knowledge

In Natural Language Processing (NLP), **language** is the primary medium of human communication, and **knowledge** is the foundation that gives meaning to this communication. For machines to understand language effectively, they must go beyond recognizing words and grammar—they must also interpret meaning using **contextual, linguistic, and world knowledge**.

Language understanding involves multiple layers of analysis, each contributing to the extraction of meaning:

1. Lexical Analysis

- **Definition:** Focuses on individual words and their morphological structure. It involves identifying word boundaries, root forms (lemmas), and parts of speech.
- **Example:**
 - Input: *"The cats are running."*
 - Lexical analysis identifies:
 - *cats* → root: *cat*, plural noun.
 - *running* → root: *run*, present participle verb.
- **Importance:** Helps in understanding grammatical roles and in normalizing word forms for further processing.

2. Syntactic Analysis (Parsing)

- **Definition:** Deals with the grammatical structure of sentences and how words combine to form valid phrases or sentences according to grammar rules.
- **Example:**
 - Sentence: *"The boy ate the apple."*
 - Parsing reveals structure:
 - Subject: *The boy*
 - Verb: *ate*
 - Object: *the apple*
- **Importance:** Determines the relationship between words, which is crucial for meaning extraction and disambiguation.

3. Semantic Analysis

- **Definition:** Concerned with the meaning of words and how they combine to form meaningful sentences. It often uses lexical resources (e.g., WordNet) and semantic roles.
- **Example:**
 - Sentence: *"The bank raised interest rates."*
 - Semantic analysis determines *bank* refers to a **financial institution**, not a riverbank.
- **Importance:** Resolves word sense ambiguity and supports meaning-based applications like question answering and information retrieval.

4. Discourse Analysis

- **Definition:** Examines how sentences relate to each other in larger text or conversation to form coherent meaning. It accounts for anaphora (reference to earlier text) and topic continuity.
- **Example:**
 - Text: *"Ravi bought a car. He drives it every day."*
 - Discourse analysis resolves that *He* refers to *Ravi* and *it* refers to *car*.
- **Importance:** Necessary for multi-sentence understanding, summarization, and dialogue systems.

5. Pragmatic Analysis

- **Definition:** Focuses on the intended meaning of sentences in specific contexts, considering speaker intent, social norms, and implied meanings.
- **Example:**
 - Sentence: *"Can you pass the salt?"*
 - Literal meaning: Asking about the listener's ability.
 - Pragmatic meaning: A polite request to hand over the salt.
- **Importance:** Enables systems to understand indirect requests, idioms, sarcasm, and cultural nuances.

Role of Knowledge in Language Understanding

To process language effectively, a computer must go beyond the surface structure of words and understand the underlying meaning, which often depends on context, background knowledge, and world understanding. For example, the sentence **"He went to the bank"** is ambiguous without knowing whether the context refers to a financial institution or a riverbank. This shows that linguistic knowledge alone is insufficient; we also need real-world knowledge to interpret meaning correctly.

Linguistic knowledge includes several layers:

- Phonology: sound patterns of language.
- Morphology: structure of words.
- Syntax: grammatical arrangement of words.
- Semantics: meaning of words and sentences.
- Pragmatics: meaning in context and intended use.

World knowledge (or extra-linguistic knowledge) encompasses facts about the real world, cultural norms, domain-specific expertise, and logical relationships. In NLP, this is

often represented using ontologies, knowledge graphs, or structured databases that link words to real-world entities and concepts.

The integration of language and knowledge is essential for tasks such as question answering, information retrieval, machine translation, and text summarization. Modern NLP systems combine linguistic rules, statistical learning, and knowledge representation techniques to interpret and generate human-like responses. Without this synergy, systems risk producing outputs that are grammatically correct but semantically or contextually incorrect.

Thus, the interplay between language and knowledge forms the foundation for building truly intelligent language-processing systems capable of meaningful human-machine interaction.

1.4 The Challenges of NLP

Natural Language Processing faces several challenges because human languages are complex, context-dependent, and often ambiguous. These challenges arise from the need to process not only the words themselves but also the meaning, structure, and intended context behind them. Below are key challenges, with emphasis on **representation and interpretation**, **semantics**, **quantifier scoping**, and **ambiguity**.

1. Representation and Interpretation

- **Description:**
In NLP, **representation** refers to how linguistic information is stored in a computer-readable format, while **interpretation** refers to extracting meaning from that representation. Choosing the right representation—whether syntactic trees, semantic networks, logic-based forms, or embeddings—is crucial for accurate understanding.
- **Example:**
 - Sentence: *“Ravi saw the man with a telescope.”*
 - Representation options:
 - **Syntactic parse 1:** Ravi used a telescope to see the man.
 - **Syntactic parse 2:** Ravi saw a man who had a telescope.
 - The chosen representation must support correct interpretation based on context.
- **Challenge:**
A wrong representation can lead to incorrect meaning extraction, especially in downstream tasks like question answering or translation.

2. Semantics

- **Description:**
Semantics deals with understanding **the meaning of words, phrases, and sentences**. In

NLP, this involves mapping linguistic forms to real-world entities, actions, and relationships.

- **Example:**

- Sentence: *"The bank raised interest rates."*
- The word *bank* could mean:
 1. Financial institution (intended meaning here).
 2. Riverbank (incorrect in this context).

- **Challenge:**

Words often have multiple meanings (**polysemy**), and resolving them requires context and background knowledge.

3. Quantifier Scoping

- **Description:**

Quantifier scoping problems occur when sentences contain quantifiers (e.g., *all*, *some*, *every*, *none*), and the logical interpretation changes depending on how these quantifiers are ordered or scoped in meaning representation.

- **Example:**

- Sentence: *"Every student read a book."*
 - Interpretation 1: There is **one specific book** that all students read.
 - Interpretation 2: Each student read **possibly different books**.

- **Challenge:**

Determining the correct scope of quantifiers requires sophisticated semantic parsing and sometimes real-world reasoning.

4. Ambiguity Issues

Ambiguity is one of the most fundamental problems in NLP. It appears at multiple levels:

1. **Lexical Ambiguity** – A single word has multiple meanings.

- Example: *"He sat on the bank."* → bank = riverbank or financial institution.

2. **Syntactic Ambiguity** – Multiple possible grammatical structures.

- Example: *"I saw the man with the telescope."* (as discussed above).

3. **Semantic Ambiguity** – Multiple interpretations of meaning despite clear grammar.

- Example: *"Visiting relatives can be annoying."* → Relatives who visit are annoying
OR the act of visiting relatives is annoying.

4. **Pragmatic Ambiguity** – Multiple interpretations due to unclear intent.

- Example: *"Can you pass the salt?"* → Question about ability or a polite request.

- **Challenge:**

Disambiguation often requires integrating **world knowledge**, **statistical models**, and **contextual cues** to arrive at the most likely interpretation.

1.5 Language and Grammar

Language is a structured system of communication, and **grammar** provides the set of rules that define how words can be combined to form valid sentences. In Natural Language Processing (NLP), grammar plays a central role in analyzing and generating language, ensuring that the system's output is both **syntactically correct** and **meaningful**.

1. Language Specifications

Language specifications describe the components and rules that define a language. In NLP, these specifications help in designing parsers, translators, and other language-processing tools. The main elements include:

- **Alphabet / Symbols:** The set of characters or tokens used in the language (e.g., English letters, punctuation marks).
- **Lexicon:** The vocabulary of the language, including words and their parts of speech.
- **Grammar Rules:** Formal specifications that describe how words can be combined.
- **Semantics:** The rules for interpreting meaning from the structures formed by grammar.

2. Different Types of Grammars in NLP

In NLP, various grammar formalisms have been developed to model the structure of natural languages more accurately than simple context-free grammars. These grammar types not only define **syntactic rules** but also integrate **semantic and functional aspects** of language. Below is an elaboration of the major grammar types.

1. Transformational Grammar (TG)

- **Origin:** Introduced by **Noam Chomsky** in the 1950s.
- **Core Idea:** Sentences have **deep structures** (underlying meaning) and **surface structures** (actual spoken/written form). Transformational rules convert deep structure into surface structure.
- **Example:**
 - Deep structure: *"John eats the apple."*
 - Transformation (passive): *"The apple is eaten by John."*
- **Use in NLP:** Helps in paraphrasing, syntax-based translation, and semantic interpretation.

2. Lexical Functional Grammar (LFG)

- **Origin:** Developed by Joan Bresnan and Ronald Kaplan in the late 1970s.
- **Core Idea:** Separates sentence structure into **two parallel representations**:

1. **C-structure (Constituent structure):** Phrase structure tree (syntactic form).
 2. **F-structure (Functional structure):** Grammatical relations (subject, object, tense).
- **Example:**
 - Sentence: *"The dog chased the cat."*
 - C-structure: NP + VP arrangement.
 - F-structure: Subject = dog, Object = cat, Verb = chased.
 - **Use in NLP:** Useful for machine translation and grammar checking, as it captures both form and function.

3. Government and Binding Theory (GB)

- **Origin:** Introduced by Noam Chomsky in the early 1980s.
- **Core Idea:** Based on the concept of **"government"** (how certain words determine the form of others) and **"binding"** (how pronouns and noun phrases are linked).
- **Example:**
 - *"John saw himself."* → *himself* refers to *John* (binding).
 - *"John saw him."* → *him* refers to another person (not John).
- **Use in NLP:** Pronoun resolution, syntactic constraints in parsing.

4. Generalized Phrase Structure Grammar (GPSG)

- **Origin:** Developed in the 1980s as an alternative to transformational grammar.
- **Core Idea:** Relies entirely on **context-free grammar rules** but enriches them with **features** (like tense, number, gender) to avoid transformations.
- **Example:**
 - Rule: $S \rightarrow NP[NUM=?n] VP[NUM=?n]$ ensures subject–verb agreement.
- **Use in NLP:** Efficient parsing, syntax validation without deep transformations.

5. Dependency Grammar (DG)

- **Origin:** Developed by Lucien Tesnière in the 1950s.
- **Core Idea:** The structure of a sentence is represented as a **set of binary relations** (dependencies) between words, with one word as the head and others as dependents.
- **Example:**
 - *"The boy eats an apple."*
 - *eats* is the root (head); *boy* (subject) and *apple* (object) depend on it.
- **Use in NLP:** Dependency parsing in search engines, question answering, and relation extraction.

6. Paninian Grammar

- **Origin:** Based on the ancient grammatical framework by **Pāṇini**, adapted for computational linguistics (especially Indian languages).
- **Core Idea:** Uses **karaka relations** (semantic roles) to represent the meaning and function of words in a sentence, independent of word order.
- **Example** (in Hindi):
 - राम ने फल खाया (Ram ne phal khaya) → Ram (agent/karaka), fruit (object/karaka).
- **Use in NLP:** Morphological analysis, free-word-order language processing (e.g., Hindi, Sanskrit).

7. Tree-Adjoining Grammar (TAG)

- **Origin:** Developed by Aravind Joshi in the late 1970s.
- **Core Idea:** Uses **elementary trees** and two operations—**substitution** and **adjunction**—to generate complex sentences.
- **Example:**
 - Elementary tree for “John eats” + adjunction of “quickly” → “John quickly eats.”
- **Use in NLP:** Machine translation, generation, and syntactic analysis due to flexibility in handling long-distance dependencies.

3. Structure of a Sentence

A sentence is not just a random sequence of words—it is an organized structure where each word plays a specific grammatical and semantic role. In **Natural Language Processing (NLP)** and **linguistics**, understanding sentence structure is essential for tasks like parsing, translation, and question answering.

At a high level, sentence structure can be viewed in two dimensions:

1. **Surface Structure** – The actual arrangement of words in a sentence (what we see or hear).
2. **Deep Structure** – The underlying meaning or conceptual representation of the sentence, independent of its exact wording.

1. Basic Components of Sentence Structure

Most English sentences can be broken down into two main parts:

- **Noun Phrase (NP):** Refers to the subject or object (e.g., *The boy, an apple*).
- **Verb Phrase (VP):** Refers to the action and its complements (e.g., *ate an apple*).

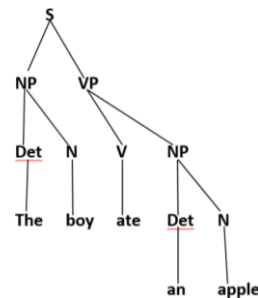
Basic Rule:

Sentence (S) → Noun Phrase (NP) + Verb Phrase (VP)

Example: “The boy ate an apple.”

- NP: *The boy* → Det (*the*) + Noun (*boy*)
- VP: *ate an apple* → Verb (*ate*) + NP (*an apple*)

Parse Tree Representation:



2. Surface Structure

- **Definition:** The actual syntactic form of the sentence—how words are arranged according to grammatical rules.
- **Example 1 (Active voice):**
 - *"The cat chased the mouse."*
- **Example 2 (Passive voice):**
 - *"The mouse was chased by the cat."*
- Both sentences have **different surface structures** but can share the same meaning in terms of who is acting and who is affected.

3. Deep Structure

- **Definition:** The underlying semantic relationships in a sentence—the "core meaning" before applying any syntactic transformations.
- **Example:**
 - Deep structure: [Subject: cat] [Action: chase] [Object: mouse]
 - Surface structure variations:
 1. *"The cat chased the mouse."* (active)
 2. *"The mouse was chased by the cat."* (passive)
- **Importance:** Deep structure is essential for NLP tasks like machine translation, where the system needs to understand meaning before generating the correct surface form.

4. Example: Surface vs. Deep Structure in Transformational Grammar

- **Deep Structure:** Conceptual representation → *John give Mary a book*
- **Transformations** to create different surface structures:
 1. Active voice: *"John gave Mary a book."*
 2. Passive voice: *"Mary was given a book by John."*

3. Question form: *"Did John give Mary a book?"*
4. Emphatic form: *"It was John who gave Mary a book."*

Transformational Grammar Overview

Transformational Grammar (TG) is a linguistic theory introduced by Noam Chomsky in the 1950s. It explains how we can generate an unlimited number of grammatically correct sentences by starting with a deep structure (basic meaning) and applying various rules to produce the surface structure (actual spoken/written form). For example, *"John gave Mary a book"* and *"Mary was given a book by John"* are two different surface structures, being generated from the same deep structure *"John give Mary a book."*

TG consists of three main components:

1. **Phrase Structure Grammar (PSG)** — generates the basic deep structure.
2. **Transformational Rules** — convert the deep structure into different surface structures.
3. **Morphophonemic Rules** — adjust the actual word forms for correct grammar and pronunciation.

1. Phrase Structure Grammar (PSG)

PSG provides the **hierarchical structure** of sentences through **phrase structure rules**. These rules define how a sentence is built from basic categories like NP (Noun Phrase) and VP (Verb Phrase). As an example, consider the following set of rules:

S → NP + VP

VP → V + NP

NP → Det + Noun

V → Aux + Verb

Det → the, a, an, this, that, these, those, my, your, his, her, its, our, their, some, any, many, few, much, every, each, etc. (Determiners)

Verb → catch, write, eat, ate, give,

Noun → boy, apple, book, marry, john, police, snatcher,

Aux → will, is, can, be, have, do,

2. Transformational Rules

Transformational rules take the **deep structure** and produce the **surface structure** by rearranging, inserting, or modifying elements.

Common transformations:

1. **Tense Addition**
 - Rule: Add past tense to "give" → "gave"

- Output: *John gave Mary a book.*

2. Passive Transformation

- Rule: Move the object to the subject position, insert "be + past participle", and add "by" before the subject.
- Output: *Mary was given a book by John.*

3. Question Formation

- Rule: Move auxiliary verb to sentence-initial position.
- Output: *Did John give Mary a book?*

4. Negation

- Rule: Insert "not" after auxiliary.
- Output: *John did not give Mary a book.*

4. Morphophonemic Rules

Morphophonemic rules determine **how morphemes change** when they combine, to produce correct **word forms** and **pronunciation**.

Example Rules:

1. Verb Past Tense Rule

- Base verb + {past} → irregular form changes
give → gave
go → went
- For regular verbs: add "-ed" (walk → walked)

2. Plural Formation Rule

- Add "-s" to nouns unless they have irregular forms.
book → books
child → children

3. Pronunciation Adjustments

- Plural "-s" is pronounced /s/ after voiceless sounds (cats), /z/ after voiced sounds (dogs), and /ɪz/ after sibilants (buses).

Applied Example:

Deep structure verb = give + {past} → Morphophonemic rule → gave

1.6 Processing Indian Languages

Indian languages span **two major families**—**Indo-Aryan** (Hindi, Urdu, Bengali, Marathi, Punjabi, etc.) and **Dravidian** (Tamil, Telugu, Kannada, Malayalam). They bring rich morphology, diverse scripts, and flexible word order that make NLP both exciting and tricky.

What makes Indian-language NLP challenging?**1. Rich morphology (data sparsity)**

- Hindi: लड़का(boy), लड़के, लड़कों, लड़के से... many surface forms → sparse counts.
- Dravidian languages are often **agglutinative**: Tamil படித்தவர்களிடம் ("from the readers") packs stem+multiple suffixes.
- **Effect**: token explosion, out-of-vocabulary; simple word-level models struggle.

2. Free(er) word order & postpositions

- Canonical order is often **SOV**: राम ने सेब खाया (Ram-ERG apple ate).
- Case expressed by **postpositions/suffixes** (Hindi में, से; Kannada -alli, -inda).
- **Effect**: dependency parsing and translation need robust, non-rigid models (e.g., dependency parsers, attention models).

3. Scripts & orthography

- Hindi uses **Devanagari** with conjuncts (e.g., क्त, र्ण), nukta (़).
- Urdu uses **Perso-Arabic Nastaliq**: cursive, right-to-left, heavy ligaturing.
- Other scripts: **Gurmukhi** (Punjabi), **Bangla**, **Gujarati**, **Odia**, **Tamil**, **Telugu**, **Kannada**, **Malayalam**—each with its own vowel signs, diacritics, and shaping rules.
- **Effect**: tokenization, OCR, rendering/normalization (Unicode NFC/NFD) and search become non-trivial.

4. Code-mixing & transliteration

- Hinglish: "kal meeting hai please reschedule karo"; Urdu-in-Roman: "aap kaisay hain?"
- **Effect**: language identification (LID), back-transliteration, mixed-script embeddings are required.

5. Named entities & transliteration variance

- Shah Rukh Khan vs शाहरुख़ ख़ान vs S.R. Khan.
- **Effect**: NER needs phonetic/graph-based matching, cross-script gazetteers.

6. Resource imbalance

- Fewer labeled corpora, treebanks, lexica compared to English.
- **Effect**: reliance on multilingual transfer, weak supervision, and synthetic data.

There are a number of **differences between Indian languages and English**. Some of these are listed here.

Aspect	English	Indian languages (typical)
Language family	Germanic (Indo-European)	Indo-Aryan & Dravidian (plus Tibeto-Burman, etc.)
Canonical word order	SVO (<i>John eats apples</i>)	Often SOV (<i>राम सेब खाता है</i>)
Morphology	Mostly analytic, light inflection	Richer inflection ; Dravidian often agglutinative
Case marking	Prepositions (to, in, by)	Postpositions/case suffixes (<i>में, से, -alli, -inda</i>)
Scripts	Latin (left-to-right)	Multiple Indic scripts + Urdu Nastaliq (RTL)
Tokenization	Whitespace + punctuation works well	Needs script-aware segmentation; conjuncts/diacritics

1.7 NLP Applications

1. Machine Translation (MT)

- **What it does:** Automatically converts text from one language to another.
- **Example:**
 - English → Hindi: "How are you?" → "आप कैसे हैं?"
 - Hindi ↔ Urdu back-transliteration for search engines.
- **Use cases:** Google Translate, IndicTrans for Indian languages, cross-border communication.

2. Sentiment Analysis & Opinion Mining

- **What it does:** Detects sentiment polarity (positive, negative, neutral) and emotional tone in text.
- **Example:**
 - "The movie was fantastic!" → Positive sentiment.
 - "सड़क की हालत खराब है!" → Negative sentiment.
- **Use cases:** Social media monitoring, product reviews, political opinion analysis.

3. Text Summarization

- **What it does:** Produces a concise version of a long text while preserving meaning.
- **Types:**
 - **Extractive:** Selects key sentences.
 - **Abstractive:** Generates new paraphrased sentences.
- **Example:** Summarizing a 10-page news article into a few bullet points for quick reading.

- **Use cases:** News aggregators, academic literature review tools.

4. Question Answering (QA)

- **What it does:** Directly answers user questions from structured/unstructured data.
- **Example:**
 - User: *"Who is the Prime Minister of India?"*
 - System: *"Narendra Modi"*
- **Variants:**
 - **Open-domain:** Pulls from general knowledge (Google Search snippets).
 - **Closed-domain:** Answers within a specific dataset (medical QA).
- **Use cases:** Chatbots, customer support, virtual assistants.

5. Information Extraction (IE)

- **What it does:** Automatically identifies entities, relationships, and events in text.
- **Example:**
 - Sentence: *"Ratan Tata was born in 1937 in Mumbai."*
 - Entity: *Ratan Tata* (Person)
 - Date: *1937*
 - Location: *Mumbai*
- **Use cases:** Building knowledge graphs, legal document analysis.

6. Named Entity Recognition (NER)

- **What it does:** Detects proper nouns such as people, locations, organizations, dates.
- **Example:**
 - *"Virat Kohli plays for Royal Challengers Bangalore in IPL."*
 - **Person:** Virat Kohli
 - **Organization:** Royal Challengers Bangalore
 - **Event:** IPL
- **Use cases:** Search engines, news indexing, financial analytics.

7. Text Classification

- **What it does:** Categorizes documents into predefined classes.
- **Example:**
 - Email classification: Spam vs Non-spam.
 - News categorization: Politics, Sports, Technology.

- **Use cases:** Email filtering, content moderation.

8. Speech Recognition (ASR)

- **What it does:** Converts spoken language into text.
- **Example:**
 - Hindi ASR: "आज मौसम अच्छा है" → "आज मौसम अच्छा है"
- **Use cases:** Voice assistants, transcription services, dictation software.

9. Text-to-Speech (TTS)

- **What it does:** Converts text into natural-sounding speech.
- **Example:**
 - Screen readers for the visually impaired.
 - Reading out news articles in regional languages.
- **Use cases:** Accessibility tools, navigation systems.

10. Optical Character Recognition (OCR)

- **What it does:** Extracts text from scanned documents or images.
- **Example:**
 - Reading Hindi from a scanned newspaper image: "भारत ने स्वर्ण पदक जीता"
- **Use cases:** Digitizing printed books, document archival.

11. Chatbots & Conversational Agents

- **What it does:** Engages users in natural language conversation.
- **Example:**
 - Banking chatbot: "Show my last 5 transactions" → Displays data.
- **Use cases:** Customer service, virtual shopping assistants.

12. Document Clustering & Topic Modeling

- **What it does:** Groups documents based on thematic similarity.
- **Example:**
 - Cluster research papers into "Machine Learning," "Cybersecurity," etc.
- **Use cases:** Search engines, academic research organization.

13. Automatic Spell Checking & Grammar Correction

- **What it does:** Detects and corrects spelling/grammar errors.
- **Example:**
 - Input: "He go to school yesterday" → Output: "He went to school yesterday"

- **Use cases:** Word processors, language learning tools.

14. Language Identification (LID)

- **What it does:** Detects the language of a given text.
- **Example:** "आप कैसे हैं?" → Hindi.
- **Use cases:** Multilingual chatbots, automatic translation pipelines.

15. Code-Mixed Language Processing

- **What it does:** Handles text mixing multiple languages.
- **Example:** "Kal meeting hai please reschedule kar dena" → Identify Hindi + English words.
- **Use cases:** Social media analysis in multilingual contexts.

16. Text Generation

- **What it does:** Produces new text given a prompt.
- **Example:**
 - AI story writing, poetry generation, news headline generation.
- **Use cases:** Creative writing tools, content automation.

17. Plagiarism Detection

- **What it does:** Finds similarities between documents.
- **Example:** Detecting copied assignments in universities.
- **Use cases:** Academic integrity, content originality checking.

18. Machine Reading Comprehension

- **What it does:** Reads a passage and answers questions about it.
- **Example:**
 - Passage: "Mahatma Gandhi led the Dandi March in 1930."
 - Q: "When did the Dandi March happen?"
 - A: "1930"
- **Use cases:** Education, automated tutoring.

19. Cross-lingual Information Retrieval

- **What it does:** Allows users to query in one language and retrieve results in another.
- **Example:** Searching in Hindi ("भारत के राष्ट्रपति") and retrieving English Wikipedia pages.
- **Use cases:** Multilingual search engines.

20. Low-resource Language Processing

- **What it does:** Brings NLP to languages with limited data (many Indian & African languages).
- **Example:** Developing lemmatizers for Manipuri or Santali.
- **Use cases:** Digital inclusion, cultural preservation.

1.8 Some Successful Early NLP Systems

1. ELIZA (1966)

- **Developer:** Joseph Weizenbaum (MIT)
- **Purpose:** Simulate a Rogerian psychotherapist through text conversation.
- **Approach:** Pattern matching and substitution.
- **Example Interaction:**
 - User: *"I feel sad today."*
 - ELIZA: *"Why do you feel sad today?"*
- **Significance:** First program to give users the illusion of understanding language.
- **Limitation:** No real understanding; responses were based purely on matching keywords and templates.

2. SYSTRAN (Started 1968)

- **Developer:** Peter Toma.
- **Purpose:** Rule-based machine translation system.
- **Initial Use:** US Air Force translated Russian scientific/technical documents into English during the Cold War.
- **Approach:** Direct translation with linguistic rules and bilingual dictionaries.
- **Example:**
 - Russian: *"Привет, мир!"* → English: *"Hello, world!"*
- **Significance:** One of the earliest commercial MT systems; still in use with modern updates.
- **Limitation:** Struggled with idioms, context, and ambiguous sentences.

3. TAUM-METEO (1977)

- **Developer:** TAUM (Traduction Automatique à l'Université de Montréal)
- **Purpose:** Translate weather reports from English to French.
- **Approach:** Domain-specific machine translation with a restricted vocabulary.
- **Example:**
 - English: *"Cloudy with occasional showers."*

- French: *"Nuageux avec quelques averses."*
- **Significance:** First fully automated, production-level MT system in daily use for years.
- **Limitation:** Domain-limited; couldn't handle general text outside weather forecasting.

4. SHRDLU (1970–1972)

- **Developer:** Terry Winograd (MIT)
- **Purpose:** Natural language understanding and command execution in a virtual "blocks world."
- **Approach:** Combined syntactic parsing, semantic interpretation, and logical reasoning within a limited environment.
- **Example:**
 - User: *"Pick up the red block and place it on the green cube."*
 - SHRDLU: Executes the action in the simulated world.
- **Significance:** Demonstrated deep understanding within a constrained domain.
- **Limitation:** Not scalable to open-domain problems.

5. LUNAR (1971)

- **Developer:** William A. Woods.
- **Purpose:** Answer questions about lunar rock samples collected during Apollo missions.
- **Approach:** Semantic grammar + database query system.
- **Example:**
 - User: *"What is the aluminum content of sample 10017?"*
 - LUNAR: Retrieves and reports the correct value from its database.
- **Significance:** High accuracy for a specific scientific dataset.
- **Limitation:** Limited to one dataset; lacked adaptability.

6. PARRY (1972)

- **Developer:** Kenneth Colby.
- **Purpose:** Simulate a patient with paranoid schizophrenia for psychiatric interviews.
- **Approach:** More sophisticated than ELIZA; used a model of beliefs, fears, and intentions.
- **Example:**
 - Psychiatrist: *"Do you trust people?"*
 - PARRY: *"I don't trust people who talk about me behind my back."*
- **Significance:** Showed how computational models can simulate human psychological patterns.

- **Limitation:** Still rule-based and unable to learn.

7. JANUS Speech-to-Speech Translation (1990s)

- **Developer:** Carnegie Mellon University & Karlsruhe University.
- **Purpose:** Real-time speech translation between English, German, and Japanese.
- **Approach:** Speech recognition → MT → Speech synthesis.
- **Significance:** Early prototype of modern Google Translate speech mode.
- **Limitation:** Accuracy dropped sharply in noisy environments.

1.9 Information Retrieval (IR)

1. What is Information Retrieval?

Information Retrieval is the process of **finding relevant information** from a large collection of unstructured data (often text-based) in response to a user query.

- The **goal** is to deliver documents or information items that satisfy the user's information need, rather than exact data matching.
- IR systems are at the core of **search engines** like Google, Bing, and academic databases.

Formal Definition:

Information Retrieval deals with the representation, storage, organization, and access to information items, typically documents, to satisfy the information needs of users.

2. Key Characteristics of IR

- **Unstructured Data:** Mostly natural language text rather than structured databases.
- **User-Centered:** The system must interpret vague, incomplete, or ambiguous queries.
- **Relevance-Oriented:** Instead of returning exact matches, IR ranks results by estimated usefulness.

3. Basic IR Process

1. **Document Representation** – Represent documents in a form suitable for indexing (e.g., using keywords, term frequency).
2. **Indexing** – Build a searchable index to speed up retrieval.
3. **Query Processing** – Accept user queries (often keywords or natural language).
4. **Matching and Ranking** – Compare query with document index and rank results.
5. **Evaluation** – Measure retrieval effectiveness using metrics like precision, recall, and F1-score.

4. Major Issues in Information Retrieval

A. Representation of Information

- **Problem:** How to represent documents and queries so that they can be compared effectively.
- **Example:** The sentence "*Apple unveils new iPhone*" might be indexed under apple, unveil, new, iphone.
- **Challenge:** Synonyms (*car* vs *automobile*), polysemy (*apple* as fruit vs company).

B. Query Formulation

- **Problem:** Users often submit vague, short, or ambiguous queries.
- **Example:** A query like "*Jaguar speed*" could mean the car or the animal.
- **Challenge:** Need query expansion techniques (e.g., adding synonyms, spelling correction).

C. Relevance Determination

- **Problem:** Different users may have different relevance judgments for the same query.
- **Example:** Searching "*Python tutorial*"—a beginner and an expert will want different levels of detail.
- **Challenge:** Designing algorithms that can predict user intent and preferences.

D. Ranking of Results

- **Problem:** Retrieving relevant documents is not enough—they must be **ranked** so the most useful ones appear first.
- **Challenge:** Using ranking algorithms like **TF-IDF**, **BM25**, or neural ranking models to balance relevance and diversity.

E. Scalability and Efficiency

- **Problem:** Modern IR systems must handle billions of documents.
- **Challenge:** Building distributed, scalable search architectures (e.g., inverted index, sharding).

F. Multilingual and Cross-Language Retrieval

- **Problem:** Users may query in one language while documents exist in another.
- **Example:** Query in Hindi: "*भारत की अर्थव्यवस्था*" → retrieve English documents on "Indian economy".
- **Challenge:** Requires machine translation and multilingual indexing.

G. Evaluation and User Feedback

- **Problem:** Determining how good the system is.
- **Challenge:** Need datasets with ground truth relevance judgments, and metrics such as:
 - **Precision:** fraction of retrieved documents that are relevant.

- **Recall**: fraction of relevant documents retrieved.
- **F1-score**: harmonic mean of precision and recall.

1.10 Language Modelling

A **Language Model (LM)** is a probabilistic framework that assigns a likelihood to a sequence of words in a language. Its goal is to capture the structure and patterns of natural language so that it can **predict the next word** in a sequence or **evaluate the probability of an entire sentence**.

Key Idea

Given a sequence of words $w_1, w_2, w_3, \dots, w_n$, a language model estimates:

$$P(w_1, w_2, w_3, \dots, w_n)$$

or more commonly, predicts:

$$P(w_n | w_1, w_2, \dots, w_{n-1})$$

Applications

- **Speech recognition** – Predicting likely word sequences.
- **Machine translation** – Scoring translation fluency.
- **Spelling correction** – Suggesting contextually correct words.
- **Text generation** – Producing coherent, human-like text.

Example

Sentence: "I love eating ____"

- The LM calculates probabilities for possible next words:
 - pizza → 0.45
 - books → 0.05
 - apples → 0.30
 - running → 0.02
- It picks "**pizza**" as the most likely continuation.

1.10.1 Various Grammar-Based Language Models

In Natural Language Processing, a **grammar-based language model** defines the structure and formation rules of valid sentences in a language. These models specify how smaller linguistic units (words, morphemes) combine into larger ones (phrases, sentences) using **formal grammar rules**. Grammar-based models differ from purely statistical models because they are **rule-driven**, focusing on syntactic correctness and structural generation rather than just probability.

1. Generative Grammars

Definition:

A generative grammar is a formal system that defines **how sentences can be generated** from a finite set of rules and symbols. Introduced by **Noam Chomsky** in the 1950s, it describes **all possible grammatical sentences** in a language.

Core Idea:

- Start from a **start symbol** (e.g., S for sentence).
- Apply production rules to rewrite symbols until only words remain.
- Each valid derivation produces a grammatical sentence.

Components

1. **Terminals** – Actual words in the language (e.g., "dog", "runs").
2. **Non-terminals** – Syntactic categories (e.g., NP for noun phrase, VP for verb phrase).
3. **Production Rules** – Rewrite patterns (e.g., $S \rightarrow NP VP$).
4. **Start Symbol** – Usually S for sentence.

Example (Phrase Structure Grammar)

Rules:

```
S → NP VP
NP → Det N
VP → V NP
Det → "the" | "a"
N → "dog" | "cat"
V → "chased" | "saw"
```

Generation Example:

```
S
→ NP VP
→ Det N VP
→ "the" N VP
→ "the" dog VP
→ "the" dog V NP
→ "the" dog chased NP
→ "the" dog chased Det N
→ "the" dog chased a cat
```

2. Hierarchical Grammar

Definition:

A hierarchical grammar organizes language rules in **multiple levels of abstraction**, showing that language is **nested and recursive**.

Key Features:

- Language elements are built **layer by layer**.
- Lower-level rules form phrases → Phrases combine into clauses → Clauses form sentences.
- Useful for modeling **complex, nested structures** in natural languages.

Example (Hierarchy)

Level 1: Words →

```
Det: "the"
N: "boy"
V: "kicked"
Det: "the"
N: "ball"
```

Level 2: Phrases →

```
NP → Det N      → "the boy"
NP → Det N      → "the ball"
VP → V NP       → "kicked the ball"
```

Level 3: Sentence →

```
S → NP VP → "the boy kicked the ball"
```

Comparison of Generative Vs Hierarchical Grammar

Aspect	Generative Grammar	Hierarchical Grammar
Goal	Generate all valid sentences	Represent sentence structure in levels
Focus	Rule-based generation	Layered, nested syntactic organization
Use in NLP	Parsing, generation, grammar check	Parsing, speech recognition, semantic mapping
Example	Chomsky's phrase structure rules	Tree-based phrase hierarchy

3. Government and Binding Theory (GB)

Definition:

Government and Binding Theory, proposed by **Noam Chomsky** in the early 1980s, is a **theoretical framework for syntax** in generative grammar. It aims to describe the **universal principles** of natural languages and explain how language variation arises from a limited set of parameters.

The theory is called *Government and Binding* because:

- **Government** refers to the syntactic relationship where one word (often a verb) controls the form of another.
- **Binding** deals with how different noun phrases (like pronouns and anaphors) relate to each other in meaning and reference.

Key Principles

1. **Universal Grammar (UG):**

All languages share a common underlying structure.

2. **Modularity:**

Syntax is built from **interrelated modules** (sub-theories) that interact.

3. **Levels of Representation:**

GB theory distinguishes between:

- **D-structure (Deep Structure)** – Captures basic syntactic relations before transformations.
- **S-structure (Surface Structure)** – After transformations like movement.
- **PF (Phonetic Form)** – The output sent to pronunciation.
- **LF (Logical Form)** – The output sent to semantic interpretation.

Phonetic Form (PF) and Logical Form (LF)

Phonetic Form (PF):

- Represents the **sound structure** of the sentence.
- Derived from S-structure through **phonological rules**.
- Relevant for **speech production** and prosody.

Example:

Deep Structure: "John will eat the apple"

PF handles stress, intonation, and pronunciation details in actual speech.

Logical Form (LF):

- Represents the **semantic structure** of the sentence.
- Contains information necessary for **meaning interpretation** (scope, reference).
- Derived from S-structure using semantic mapping rules.

Example (Ambiguity):

"Everyone loves someone."

- LF1: For every person, there exists someone they love. ($\forall x \exists y \text{ Love}(x,y)$)
- LF2: There is one person loved by everyone. ($\exists y \forall x \text{ Love}(x,y)$)

LF helps capture such differences in meaning.

Main Components of GB Theory

Government and Binding (GB) theory, introduced by Noam Chomsky in the early 1980s, proposes that syntax is governed by Universal Grammar principles, organized into interconnected modules. Each module deals with a specific syntactic phenomenon but interacts with others to form a complete grammatical system. The following are the main components of GB Theory.

1. X-bar Theory
2. Theta (θ) Theory
3. Case Theory
4. Binding Theory,
5. Government Theory,
6. Bounding Theory, and
7. Control Theory

1. X-Bar Theory – Overview

X-Bar theory is a **universal template for phrase structure**. It was introduced to **capture generalizations about how phrases are formed in all human languages** without having to write separate phrase structure rules for every phrase type (like NP, VP, PP, AP, etc.).

Instead of writing rules such as:

- $\text{NP} \rightarrow \text{Det N}$
- $\text{VP} \rightarrow \text{V NP}$
- $\text{PP} \rightarrow \text{P NP}$

We have a **single universal schema** that works for all categories.

Core Idea

Every phrase (XP) has:

1. **Head (X)** – the core lexical item that determines the phrase type.
2. **Complement** – an element that completes the meaning of the head.
3. **Specifier** – an optional element that modifies the head phrase.
4. **Adjunct** – optional modifiers added recursively.

The X-Bar Schema

The theory uses **three levels** for each phrase:

1. **X⁰ (Head)** → The main word (noun, verb, adjective, preposition).
2. **X' (X-bar)** → The head plus its complement(s).
3. **XP (X phrase)** → The full phrase with specifier and possibly adjuncts.

Universal Rules:

1. **XP → Specifier X'**
2. **X' → X' Adjunct** or **X' → Adjunct X'** (depending on language order)
3. **X' → X Complement**

Example – Noun Phrase (NP)

Sentence: "The big dog"

Structure:

- Head (N) → dog
- Specifier → The
- Adjunct → big

X-Bar Representation:

```
NP → Specifier (Det) + N'
N' → Adjunct (Adj) + N'
N' → N (dog)
```

Example – Verb Phrase (VP)

Sentence: "ate the cake"

Structure:

- Head (V) → ate
- Complement → NP (the cake)
- No specifier here (subject is outside VP)

X-Bar Representation:

VP → V'

V' → V (ate) + NP (the cake)

Example: English vs. Japanese (Head Direction)

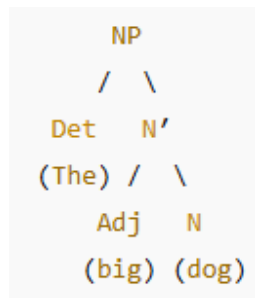
Sentence meaning: "Ate the cake"

- **English (Head-Initial):** [V ate] [NP the cake] → **ate the cake**
- **Japanese (Head-Final):** [NP keeki-o] [V tabeta] → **keeki-o tabeta**

Same X-Bar structure applies; only head direction changes.

Visual X-Bar Diagram (Example)

For "The big dog":



2. Theta Theory

Theta Theory deals with the **assignment of thematic roles** (also called **θ-roles**) to the arguments of a predicate (usually a verb). It ensures that **each argument in a sentence gets the correct semantic role** and that **each θ-role is assigned exactly once**.

In simpler terms:

- Verbs (and other predicates) have **semantic expectations** about what kinds of participants they need.
- Theta Theory formalizes these expectations and checks that they are satisfied.

Thematic Roles (θ-Roles)

Common θ-roles include:

Role	Description	Example
Agent	Doer of the action	<i>John</i> opened the door.
Theme (or Patient)	Entity affected by the action	John opened <i>the door</i> .
Experiencer	Entity experiencing a state	<i>Mary</i> felt happy.
Goal	Endpoint of movement	He went <i>to the park</i> .
Source	Origin of movement	She came <i>from Delhi</i> .
Instrument	Means used to do something	He cut it <i>with a knife</i> .
Beneficiary	Person benefiting from the action	She baked a cake <i>for him</i> .

The Theta-Criterion

A core rule of θ-theory is the **Theta-Criterion**:

Each argument in a sentence must be assigned exactly one θ-role, and each θ-role of a predicate must be assigned to exactly one argument.

Formally:

- One role per argument** → No argument can have two roles from the same predicate.
- One argument per role** → No two arguments can share the same role from the same predicate.

Example

Sentence: "**John gave Mary a book.**"

- Predicate: **give** (requires three θ -roles)
 - Agent $\rightarrow$ John (the giver)
 - Goal $\rightarrow$ Mary (the receiver)
 - Theme $\rightarrow$ a book (the thing given)

If we violate the θ -criterion:

- "*John gave Mary.*" $\rightarrow$ Missing **Theme** $\rightarrow$ Ungrammatical (incomplete theta grid).
- "*John gave Mary a book a pen.*" $\rightarrow$ Too many Themes for one predicate $\rightarrow$ Violates θ -criterion.

Theta Grid

A **Theta Grid** is a way to represent the θ -roles a verb assigns.

For **give**: give: <Agent, Goal, Theme>

In "John gave Mary a book":

- Agent = John
- Goal = Mary
- Theme = a book

Role in Sentence Structure

Theta Theory interacts with **X-Bar Theory**:

- Arguments (complements, specifiers) are positioned according to the θ -role they fulfill.
- **Complement position** $\rightarrow$ Often assigned Theme.
- **Specifier position (subject)** $\rightarrow$ Often assigned Agent or Experiencer.

Example with Movement

Sentence: "**The cake was eaten by John.**"

- Deep structure: *John ate the cake.*
 - Agent = John
 - Theme = the cake
- Surface structure changes (passive voice), but **theta roles stay the same.**

3. Case Theory

Case Theory deals with the distribution of abstract Case to noun phrases (NPs) in a sentence. It explains why certain sentences are grammatical and others are not, based on whether every overt NP receives Case. In GB, Case (with a capital "C") is an abstract syntactic feature, not just

the morphological form you see in pronouns like *he/him* or *she/her*. Even in English (where nouns don't inflect much), Case still exists at a syntactic level.

Core Principles

Case Theory enforces the **Case Filter**:

Case Filter: Every overt NP must have (and receive) abstract Case.

*NP→Case

If an NP has no Case, the sentence is ungrammatical.

Types of Case

In GB theory, Case is assigned under specific configurations:

1. Nominative Case

- Assigned by finite INFL (Tense/Agreement) to the subject NP in Spec-IP position.
- Example:
 - He eats apples. → *He* gets nominative Case from finite INFL (*eats* is present tense).

2. Accusative Case

- Assigned by transitive verbs to their object in complement position.
- Example:
 - I saw her. → *Her* gets accusative Case from the verb *saw*.

3. Oblique Case

- Assigned by prepositions.
- Example:
 - We went to him. → *Him* gets oblique Case from the preposition *to*.

Example Sentences

Grammatical (Case assigned properly)

1. John likes Mary.

- *John* → nominative Case (from finite INFL).
- *Mary* → accusative Case (from verb *likes*).

2. I gave the book to Sarah.

- *I* → nominative Case (from INFL).
- *Sarah* → oblique Case (from preposition *to*).

Binding Theory, Government Theory, Bounding Theory, and Control Theory

In the Government and Binding (GB) framework, several interrelated sub-theories explain the structural relations in syntax. **Binding Theory** governs the distribution of anaphors (e.g., *himself*), pronouns (e.g., *he*), and referential expressions (e.g., *John*) through principles A, B, and C, which determine how and where these elements can be co-referential within a sentence. **Government Theory** defines the relationship between a head (like a verb, preposition, or inflection) and the constituents it governs, influencing Case assignment and structural licensing of phrases. **Bounding Theory** restricts the distance over which elements, such as wh-phrases, can move, using bounding nodes (e.g., NP, CP) to prevent ungrammatical long-distance dependencies. **Control Theory** addresses how certain non-finite clauses have implicit subjects whose reference is determined by another element in the sentence—either the subject (subject control) or the object (object control). Together, these theories provide a unified explanation of syntactic constraints, dependencies, and movement within natural language.

4. Lexical Functional Grammar (LFG) Model

Lexical Functional Grammar (LFG) is a constraint-based grammatical framework developed by Joan Bresnan and Ronald Kaplan in the late 1970s, designed to represent linguistic structure through parallel levels of representation without transformational rules. LFG separates **constituent structure (c-structure)**, which captures the surface phrase structure of a sentence, from **functional structure (f-structure)**, which represents grammatical functions like subject, object, tense, and agreement in an abstract, language-independent form. The **lexicon** plays a central role by specifying both syntactic and semantic information for each word, linking c-structure and f-structure through lexical entries. For example, in the sentence *John gave Mary a book*, the c-structure shows a phrase tree with NP (John) as subject, NP (Mary) as indirect object, and NP (a book) as direct object, while the f-structure encodes functions such as **SUBJ = John**, **OBJ = a book**, and **OBJ θ (goal) = Mary**. This separation makes LFG well-suited for modeling languages with free word order, such as Hindi (*राम ने सीता को किताब दी*), where functional roles remain constant despite variations in constituent order. LFG's declarative nature and parallel structure representations make it particularly useful in computational linguistics and NLP applications like parsing, grammar checking, and machine translation.

C-Structure (constituent structure) and F-Structure (functional structure) in LFG Model

In the Lexical Functional Grammar (LFG) model, sentence structure is represented using two complementary levels: C-Structure (constituent structure) and F-Structure (functional structure). C-Structure captures the hierarchical arrangement of words into syntactic units (constituents) such as noun phrases (NP) and verb phrases (VP), following phrase structure rules. It reflects the surface word order of the sentence. For example, in "*John gave Mary a book*", the C-Structure shows $S \rightarrow \text{NP_Subj VP}$, with "*John*" as the subject NP, and the VP containing the verb "*gave*", an indirect object NP ("*Mary*"), and a direct object NP ("*a book*").

On the other hand, F-Structure focuses on the abstract grammatical functions and relationships independent of surface word order. It encodes attributes like PRED (predicate), SUBJ (subject), OBJ (direct object), OBJ θ (indirect object), TENSE, and more. For the same

example, the F-Structure may represent PRED: 'give <SUBJ, OBJ, OBJθ>', SUBJ: 'John', OBJ: 'book', OBJθ: 'Mary', and TENSE: past. This separation allows LFG to handle cross-linguistic variations—where the same F-Structure can be expressed by different C-Structures—making it especially powerful for languages with free or flexible word order like Hindi, as opposed to fixed word order languages like English.

5. Paninian Framework

The **Paninian Framework** is a rule-based linguistic model developed for analyzing Indian languages, originally based on the ancient Sanskrit grammar of **Pāṇini** (circa 4th century BCE). Pāṇini's *Ashtadhyayi* is considered one of the most sophisticated grammatical treatises ever written, containing nearly 4,000 sutras (rules) that describe the phonology, morphology, and syntax of Sanskrit in a precise and formalized way. The **Paninian approach** emphasizes the relationship between **karaka roles** (semantic–syntactic roles of noun phrases, similar to thematic roles in modern linguistics) and the verb, rather than strictly depending on word order.

In this framework, the **verb** is the central element of a sentence, and all other words are related to it through **karakas** such as:

- **Karta** (doer/agent) – who performs the action
- **Karma** (object/patient) – what is affected by the action
- **Karana** (instrument) – means by which the action is done
- **Sampradana** (recipient/beneficiary) – to/for whom the action is done
- **Apadana** (source) – where the action originates from
- **Adhikarana** (location) – where the action takes place

Unlike English, where **word order** primarily indicates grammatical roles, the Paninian model uses **case markers (vibhaktis)** to indicate the role of a noun phrase in a sentence. This makes it highly suitable for **free word order languages** such as Hindi, Marathi, Sanskrit, and other Indian languages.

Example in Hindi:

राम ने सीता को किताब दी (Rām ne Sītā ko kitāb dī)

- **Karta** (Agent): राम ने (Rām ne – 'Ram' with ergative marker)
- **Sampradana** (Recipient): सीता को (Sītā ko – 'to Sita')
- **Karma** (Object): किताब (kitāb – 'book')
- **Verb (action)**: दी (dī – 'gave', feminine singular past)

Even if the words are rearranged (e.g., *सीता को राम ने किताब दी*), the meaning remains the same because the **case markers** (ने, को) define the roles. This is different from English "*Ram gave Sita a book*", where changing word order could alter the meaning. In **NLP**, the Paninian framework is widely applied for **parsing Indian languages**, **machine translation**, **morphological analysis**, and **semantic role labeling**, as it naturally accommodates their rich morphology and flexible syntax.

1.10.2 Statistical Language Model

A Statistical Language Model (SLM) is a probabilistic framework that assigns a probability to a sequence of words in a language. It is widely used in speech recognition, machine translation, spelling correction, text prediction, and other NLP applications. The main idea is to estimate the likelihood of a sentence based on patterns learned from a large corpus of text.

The probability of a word sequence $W = w_1, w_2, \dots, w_n$ is given by:

$$P(W) = P(w_1) \cdot P(w_2|w_1) \cdot P(w_3|w_1, w_2) \dots P(w_n|w_1, \dots, w_{n-1})$$

Types of Statistical Language Models

1. **Unigram Model:** The probability of each word is independent of the previous words.

$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i)$$

Example: Sentence: I like apples

Suppose:

- $P(I) = 0.20$
- $P(\text{like}) = 0.10$
- $P(\text{apples}) = 0.05$

Then: $P(I \text{ like apples}) = 0.20 \times 0.10 \times 0.05 = \mathbf{0.001}$

Advantage: Very simple.

Disadvantage: Ignores context.

2. **Bigram Model:** Each word depends only on the immediately preceding word.

$$P(w_1, w_2, \dots, w_n) = P(w_1) \prod_{i=1}^n P(w_i|w_{i-1})$$

Example: Sentence: I like apples

Suppose:

- $P(I) = 0.20$

- $P(\text{like} \mid \text{I}) = 0.60$
- $P(\text{apples} \mid \text{like}) = 0.70$

Then: $P = 0.20 \times 0.60 \times 0.70 = \mathbf{0.084}$

The bigram model captures local context better than the unigram model.

3. **Trigram Model:** Each word depends on the previous two words.

$$P(w_i \mid w_{i-2}, w_{i-1})$$

Example: Sentence: I like red apples

The probability is estimated as: $P(\text{I}) \times P(\text{like} \mid \text{I}) \times P(\text{red} \mid \text{I}, \text{like}) \times P(\text{apples} \mid \text{like}, \text{red})$

Trigram models generally produce more accurate predictions but require much larger datasets.